FACULTAD DE INGENIERÍA

CARRERA PROFESIONAL DE INGENIERÍA CIVIL.

Método del Elemento Finito para Ecuaciones Diferenciales Parciales

Autor(es):

- Barboza Livaque, Yuvixa
- Chavarry Grados, Diana Carolina
- Moreno Guevara ,Mario
- Zegarra Chavez, Andy
- Salazar Saucedo ,Jose
- Vásquez Bernal , Yanina
- Vásquez Bernal , Shirley
- Salas Torres, Michael Alexander

Curso:

- Calculo IV

Docente:

- Gonzales Aredo, Yessica Judith

N° de Equipo: 5

Clase: 3449

CAJAMARCA – PERÚ

2017-1

I. TÍTULO DEL PROYECTO:

Método del Elemento Finito para Ecuaciones Diferenciales Parciales

II. PLANTEAMIENTO DEL PROBLEMA:

¿Cómo se desarrolla el método del elemento finito de un algoritmo computacional para encontrar la solución de ecuaciones diferenciales parciales el cual se utilizara el programa MATLAB?

¿Cuál es el desarrollo del algoritmo del elemento finito para la solución de ecuaciones diferenciales parciales?

¿Cómo se compilan los aspectos teóricos para desarrollar el algoritmo del método del elemento finito para la solución de ecuaciones diferenciales parciales

Las soluciones analíticas o exactas para las ecuaciones diferenciales parciales requieren de un gran esfuerzo y complejidad matemática. En la actualidad muchos de estos problemas se resuelven utilizando un software de computadora. La asignatura de cálculo, trata problemas sobre ecuaciones diferenciales parciales y utiliza algunos métodos, como los de diferencias finitas que son los más difundidos y en menor escala utilizan los elementos finitos.

El método del elemento finito nos da una alternativa que es más adecuada para hallar la solución de una ecuación diferencial parcial, pero si el desarrollo del problema se realiza en forma analítica representa un largo y tedioso trabajo. Lo ideal es desarrollar un programa que permita hallar la solución del problema.

Si utilizamos un programa del método del elemento finito optimizamos el proceso de hallar la solución de una ecuación diferencial. Este trabajo muestra un programa que encuentra la solución de una ecuación diferencial utilizando el método del elemento finito (MATLAB).

III. OBJETIVOS:

Objetivo General

Utilizar el programa MATLAB para implementar el algoritmo del método del elemento finito para la solución de ecuaciones diferenciales parciales.

Objetivos Específicos

Mostrar analíticamente el desarrollo del algoritmo del elemento finito para la solución de ecuaciones diferenciales parciales.

Utilizar el programa MATLAB para que optimice el tratamiento de datos en la solución de una ecuación diferencial parcial.

IV. MODELO MATEMÁTICO

Método del elemento finito

Para resolver ecuaciones diferenciales parciales mediante el método de elemento finito, el dominio de la solución se divide en una malla con puntos discretos o nodos. Entonces se aplica la EDP en cada nodo, donde las derivadas parciales se remplazan por diferencias finitas divididas.

V. ALGORITMO COMPUTACIONAL:

El software MATLAB

MATLAB es un medio computacional técnico, con un gran desempeño para el cálculo numérico computacional y de visualización. MATLAB integra análisis numérico, matrices, procesamiento de señales y gráficas. El ambiente de desarrollo de Interfaz de Usuario en MATLAB (GUIDE), contiene un conjunto de herramientas para crear interfaces graficas muy parecidas a las aplicaciones Windows. Estas herramientas simplifican el proceso de creación y de programación.

Discretización del dominio

Este paso consiste en dividir el dominio de la solución en elementos finitos. Los puntos de intersección de las líneas que forman los lados de los elementos se conoce como nodos, y los mismos lados se denominan líneas o planos nodales.

Ensamble

Una vez obtenidas las ecuaciones de elementos individuales, estas deben unirse o ensamblarse para caracterizar el comportamiento de todo el sistema. El proceso de ensamble está regido por el concepto de continuidad.

Descripción del problema

Antes de la era de las computadoras digitales, se dependía de las soluciones analíticas o exactas para las ecuaciones diferenciales parciales. Dichas soluciones requerían de un gran esfuerzo y complejidad matemática para hallar la solución de una ecuación diferencial parcial, como por ejemplo el método de separación de variables.

Método de separación de variables

Las ecuaciones diferenciales parciales se clasifican en dos tipos, lineal y no lineal. Si consideramos, por ejemplo, dos variables independientes x y y la variable dependiente U, una ecuación lineal tiene la forma.

$$\emptyset(D_x; D_y)U = F(x, y) \qquad (1.1)$$

donde el operador $\emptyset(D_x; D_y)$ es un polinomio en los dos operadores

$$D_x \equiv \frac{\partial}{\partial x}; \quad D_y \equiv \frac{\partial}{\partial y} \qquad (1.2)$$

Teniendo coeficientes que son funciones de las variables independientes x y y solamente. Si estos coeficientes son constantes, llamamos la ecuación una ecuación lineal con coeficientes constantes; en otro caso es una ecuación con coeficientes variables. Una ecuación diferencial parcial no lineal, es una que no es lineal.

Los elementos finitos tienen formas geométricas sencillas (triángulos, rectángulos, tetraedros, prismas, etc.) que se unen entre sí en unos puntos llamados nodos. Dentro de cada elemento las variables dependientes se interpolan a partir de sus valores en los nodos.

VI. EJERCICIOS:

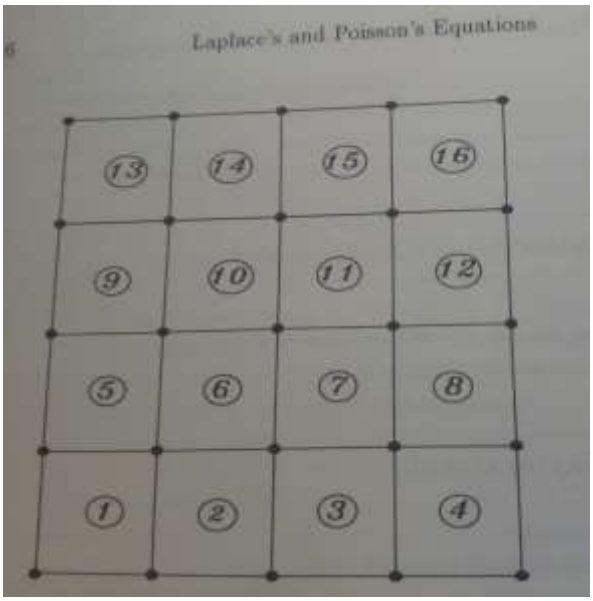
EJERCICIO N°01:

Queremos resolver la ecuación bidimensional de la place para las siguientes condiciones
El dominio y la discretización de elemento finito se muestran en la figura

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0$$

Las condiciones límite son 0 < x < 5 y 0 < y < 10, las condiciones de contorno son u(x, 0)=0, son 0 < x < 5, u (0, y)=0, 0 < y < 10, u (x, 10)=100sin (πx/10), 0 < x < 5, y $\frac{\partial u(5,y)}{\partial x} = 0$

El programa principal matlab junto con programas de funciones se enumeran a continuación. Algunos programas de función enumerados en capítulos anteriores no se enumeran aquí. Apéndice a una lista de todos los archivos de función.



- PASO N°01: Ingresamos el primer programa o programa general en cual llamaremos “Untitled7” , este nos ayudara a obtener la solución general del ejercicio así como también apoyarse de los siguientes programas.

```
a) PRIMER PROGRAMA.
nel=32;
nnel=3;
ndof=1;
nnode=25;
sdof=nnode*ndof;
gcoord(1,1)=0.0; gcoord(1,2)=0.0;
gcoord(2,1)=1.25; gcoord(2,2)=0.0;
gcoord(3,1)=2.5; gcoord(3,2)=0.0;
gcoord(4,1)=3.75; gcoord(4,2)=0.0;
gcoord(5,1)=5.0; gcoord(5,2)=0.0;
gcoord(6,1)=0.0; gcoord(6,2)=2.5;
gcoord(7,1)=1.25; gcoord(7,2)=2.5;
gcoord(8,1)=2.5; gcoord(8,2)=2.5;
gcoord(9,1)=3.75; gcoord(9,2)=2.5;
gcoord(10,1)=5.0; gcoord(10,2)=2.5;
gcoord(11,1)=0.0; gcoord(11,2)=5.0;
```

```
nodes(1,1)=1; nodes(1,2)=2; nodes(1,3)=7;
nodes(2,1)=2; nodes(2,2)=3; nodes(2,3)=8;
nodes(3,1)=3; nodes(3,2)=4; nodes(3,3)=9;
nodes(4,1)=4; nodes(4,2)=5; nodes(4,3)=10;
nodes(5,1)=1; nodes(5,2)=7; nodes(5,3)=6;
nodes(6,1)=2; nodes(6,2)=8; nodes(6,3)=7;
nodes(7,1)=3; nodes(7,2)=9; nodes(7,3)=8;

bcdof(1)=1;
bcval(1)=0;
bcdof(2)=2;
bcval(2)=0;
bcdof(3)=3;
bcval(3)=0;
bcdof(4)=4;
bcval(4)=0;
bcdof(5)=5;
```

```
ff=zeros(sdof,1);
kk=zeros(sdof,sdof);
index=zeros(nnel*ndof,1);

for iel=1:nel

nd(1)=nodes(iel,1);
nd(2)=nodes(iel,2);
nd(3)=nodes(iel,3);
x1=gcoord(nd(1),1); y1=gcoord(nd(1),2);
x2=gcoord(nd(2),1); y2=gcoord(nd(2),2);
x3=gcoord(nd(3),1); y3=gcoord(nd(3),2);

index=feeldof(nd,nnel,ndof);

k=felp2dt3(x1,y1,x2,y2,x3,y3);

kk=feasmbl1(kk,k,index);
end

[kk,ff]=feaply=2(kk,ff,bcdof,bcval);

fsol=kk\ff;

for i=1:nnode
x=gcoord(i,1); y=gcoord(i,2);
esol(i)=100*sinh(0.31415927*y)*sin(0.31415927*x)/sinh(3.1415927);
end

num=1:1:sdof;
store=[num' fsol esol']
```

Fuente: Matlab

- PASO N°02: creamos un programa el cual tendrá como nombre "`feasmbl1`", copiamos el programa y lo guardamos.

SEGUNDO PROGRAMA:

```
function [kk]=feasmbl1(kk,k,index)
edof=length(index);
for i=1:edof
ii=index(i);
for j=1:edof
jj=index(j);
kk(ii,jj)=kk(ii,jj)+k(i,j);
end
end
```

Fuente: Matlab

- PASO N°03: Insertamos el programa número 3, el cual tiene como nombre "`feaplyc2`", copiamos y guardamos.

TERCER PROGRAMA:

```
function[index]=feeldof(nd,nnel,ndof)
edof=nnel*ndof;
k=0;
for i=1:nnel
start=(nd(i)-1)*ndof;
for j=1:ndof
k=k+1;
index(k)=start+j;
end
end
```

Fuente: Matlab

- PASO N°04: Insertamos el programa número 4, el cual tiene como nombre "felp2dt3", copiamos y guardamos.

D) CUARTO PROGRAMA

```
function [k]=felp2dt3(x1,y1,x2,y2,x3,y3)
A=0.5*(x2*y3+x1*y2+x3*y1-x2*y1-x1*y3-
x3*y2);
k(1,1)=((x3-x2)*(x3-x2)+(y2-y3)*(y2-
y3))/(4*A);
k(1,2)=((x3-x2)*(x1-x3)+(y2-y3)*(y3-
y1))/(4*A);
k(1,3)=((x3-x2)*(x2-x1)+(y2-y3)*(y1-
y2))/(4*A);
k(2,1)=k(1,2);
k(2,2)=((x1-x3)*(x1-x3)+(y3-y1)*(y3-
y1))/(4*A);
k(2,3)=((x1-x3)*(x2-x1)+(y3-y1)*(y1-
y2))/(4*A);
k(3,1)=k(1,3);
k(3,2)=k(2,3);
k(3,3)=((x2-x1)*(x2-x1)+(y1-y2)*(y1-
y2))/(4*A);
```

Fuente: Matlab

- PASO N°05: Insertamos el programa número 5, el cual tiene como nombre "feaplyc2", copiamos y guardamos.

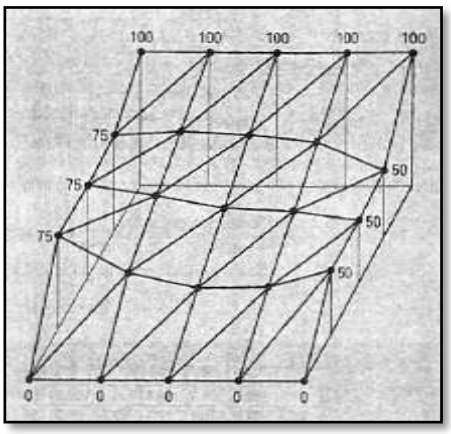
QUINTO PROGRAMA:

```
function
[kk,ff]=feaplyc2(kk,ff,bcdof,bcval)
n=length(bcdof);
sdof=size(kk);
for i=1:n
    c=bcdof(i);
    for j=1:sdof
        kk(c,j)=0;
    end
    kk(c,c)=1;
    ff(c)=bcval(i);
end
```

Fuente: Matlab

EJERCICIO N°02:

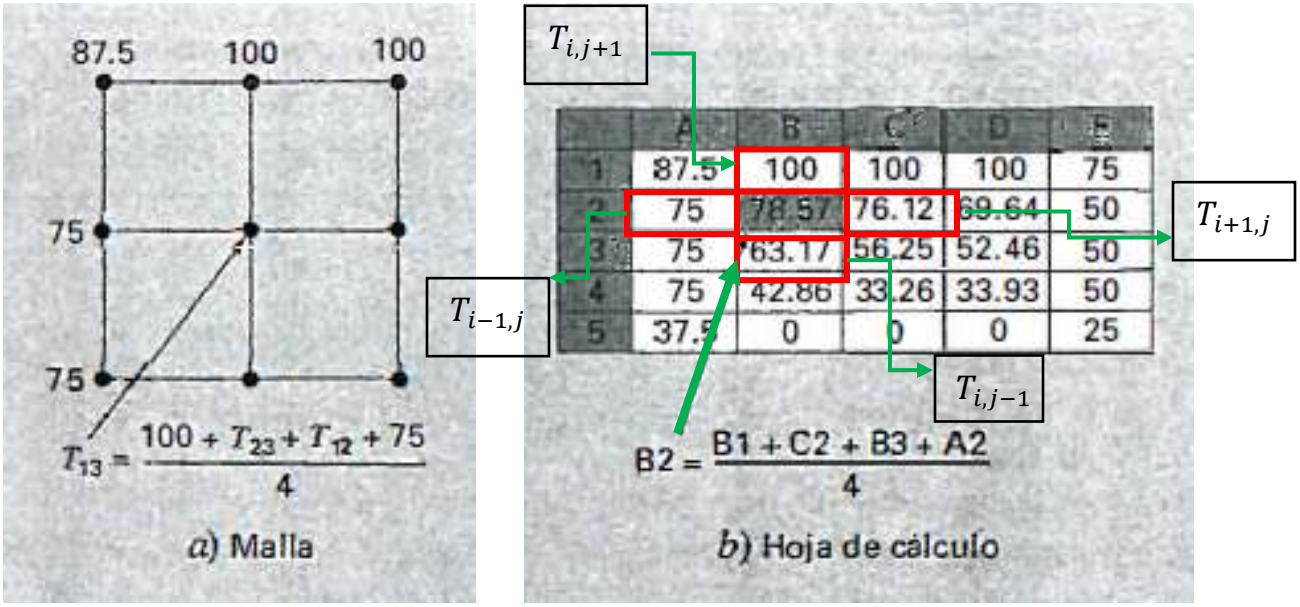
- PASO N°01: Distribución de temperatura en una placa caliente, calculada con el método del elemento finito.



Se identifica la temperatura con la fórmula de Liebman.

$$T_{i,j} = \frac{T_{i,j+1} + T_{i+1,j} + T_{i,j-1} + T_{i-1,j}}{4}$$

Esta fórmula nos permitirá calcula la temperatura en cada nodo



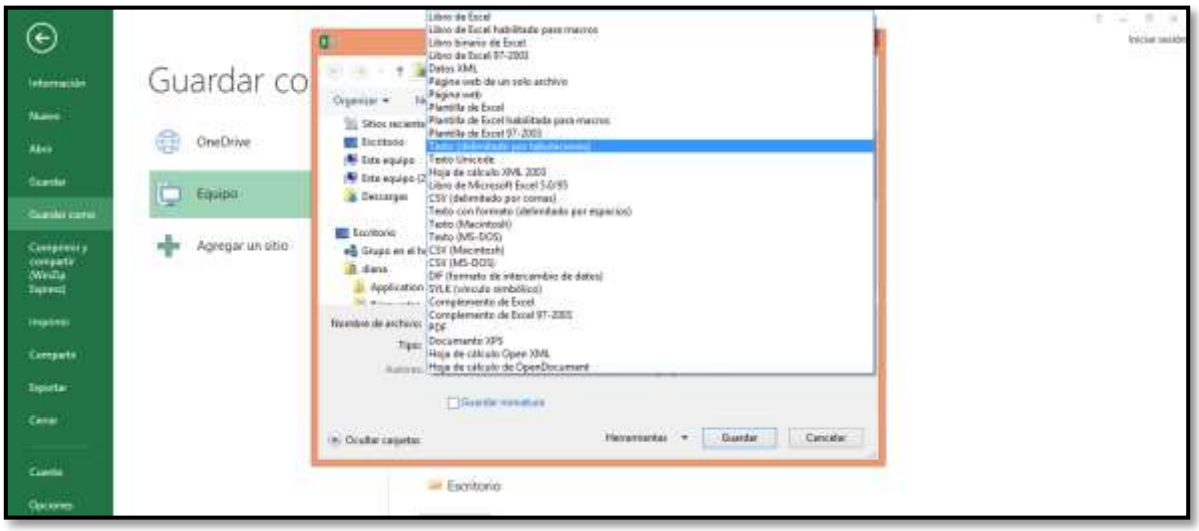
	A	B	C	D	E
1	87.5	100	100	100	75
2	75	78.57	76.12	69.64	50
3	75	63.17	56.25	52.46	50
4	75	42.86	33.26	33.93	50
5	37.5	0	0	0	25

PASO N°02: Desarrollamos la hoja de cálculo en Excel para todos los nodos

- Se usa una malla fina.
- Se aisló la frontera inferior
- Se agregó una fuente de calor de 150 a la mitad de una placa

	A	B	C	D	E	F	G	H	I
1	87.5	100.0	100.0	100.0	100.0	100.0	100.0	100.0	75.0
2	75.0	89.20	95.80	99.10	99.70	96.60	89.90	77.60	50.0
3	75.0	86.20	94.70	100.90	103.10	96.70	85.50	70.30	50.0
4	75.0	85.70	96.10	106.70	115.30	101.40	85.20	68.20	50.0
5	75.0	85.50	97.40	114.30	150.00	108.60	85.60	67.30	50.0
6	75.0	84.00	93.40	103.40	111.60	97.40	81.30	65.60	50.0
7	75.0	82.20	88.90	94.20	95.60	88.10	76.60	63.60	50.0
8	75.0	80.90	85.90	88.90	88.40	82.80	73.50	62.20	50.0
9	75.0	80.40	84.90	87.30	86.30	81.10	72.40	61.70	50.0

- PASO N°03: Se guardan este archivo, con un nombre como ***plate.txt***



PASO N°04: Trasladamos el archivo al directorio de Matlab

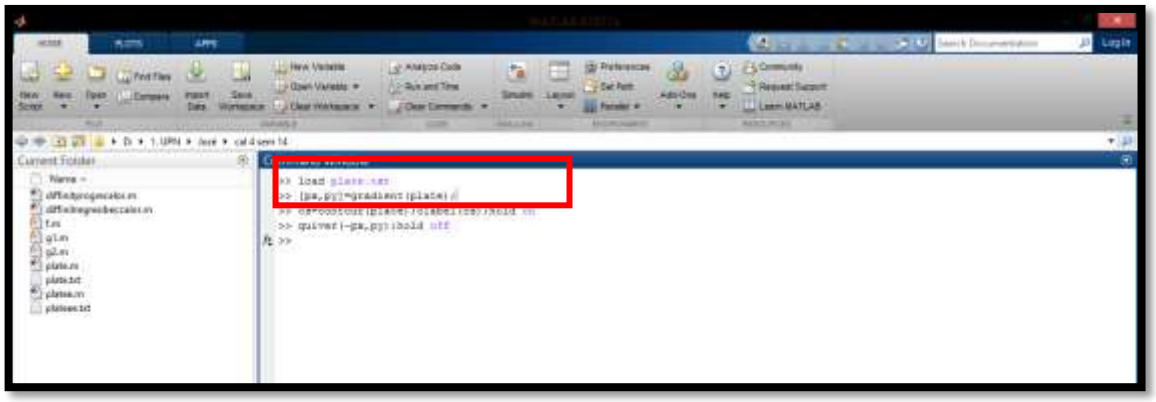
- Escribimos :

  >> load plate.txt

- Luego , los gradientes se calculan simplemente así :

  >>[px,py=gradiente (plate)]

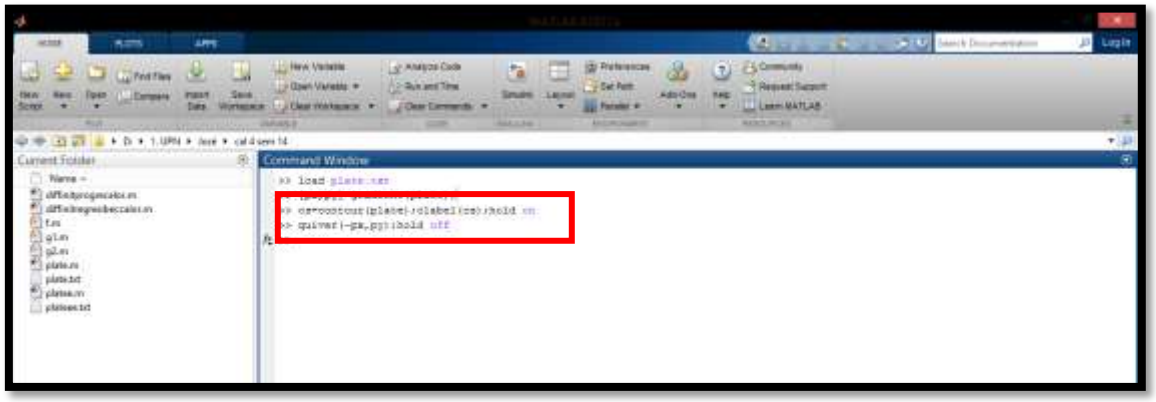
Se observa que este método es el más simple para calcular gradientes usando valores por omisión de dx=dy=1 .Por lo tanto ,serán correctas las direcciones y magnitudes relativas



Fuente: Matlab

- PASO N°05: Por último se utiliza una seria de comandos para obtener la gráfica .El comando ***contour*** desarrollo desarrollara una gráfica de contorno de los datos .El comando ***clabel*** agrega etiquetas de contorna a la gráfica. Finalmente ***quiver*** toma los datos de la gradiente y los añade a la gráfica en forma de flechas

Fuente: Matlab



VII. RESULTADOS E INTERPRETACIÓN :

EJERCICIO N°1

Imagen: Resultados luego de procesar el ejercicio N°01

```
Command Window

>> Untitled7

store =

    1.0000         0         0
    2.0000         0         0
    3.0000         0         0
    4.0000         0         0
    5.0000         0         0
    6.0000    0.0000         0
    7.0000    3.0516    2.8785
    8.0000    5.6386    5.3187
    9.0000    7.3672    6.9492
   10.0000    7.9742    7.5218
   11.0000    0.0000         0
   12.0000    7.9615    7.6257
   13.0000   14.7109   14.0904
   14.0000   19.2207   18.4100
   15.0000   20.8043   19.9268
   16.0000   -0.0000         0
   17.0000   17.7196   17.3236
   18.0000   32.7416   32.0099
   19.0000   42.7789   41.8229
   20.0000   46.3036   45.2688
   21.0000         0         0
   22.0000   38.2683   38.2683
   23.0000   70.7107   70.7107
   24.0000   92.3880   92.3880
   25.0000  100.0000  100.0000

fx >>
```

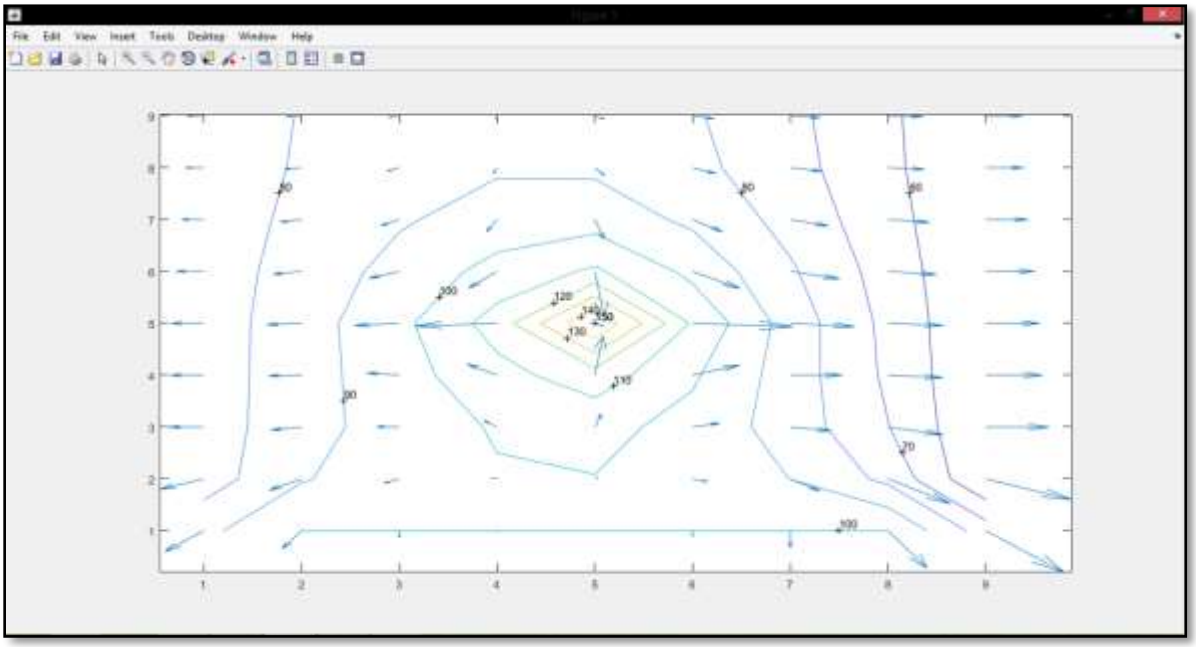
Fuente: Matlab

Interpretación:

El Matlab compara las soluciones analíticas y las soluciones hechas por el mismo programa.

EJERCICIO N°2

Imagen: Resultados luego de procesar el ejercicio N°02



Fuente: Matlab

Interpretación:

El calor fluye hacia abajo desde la fuente hasta las fronteras formando una imagen parecida a una montaña .El calor tambien fluye hacia abajo desde la frontera con temperatura alta hasta los dos extremos laterales .Se observa como el calor fluye hacia el extremo de baja temperatura. Por último se observa como la gradiente de temperatura en la dimensión y tiende a cero para el extremo inferior aislado.

VIII. APLICACIONES

La simulación mediante MEF se ha utilizado en el sector de la salud, en específico en la acción del fluido (sangre) en las arterias coronarías, remodelación y crecimiento del tejido de los vasos sanguíneos, aspectos fisiológicos y clínicos en sentido general; funcionamiento del corazón al insertar marcapasos etc. Otras de sus múltiples aplicaciones la podemos encontrar en la ingeniería civil, como por ejemplo comportamiento de suelos, cimentaciones, estructuras de hormigón etc. De igual forma en la industria mecánica en diversos sectores, siendo este programa de mucha utilidad para la verificación de cálculos en diseño automotriz, deformaciones en tanques, recipientes a presión, tuberías, vigas, grúas, en la aviación etc. como lo muestran las figuras siguientes:

<u>En el sector de la salud:</u>

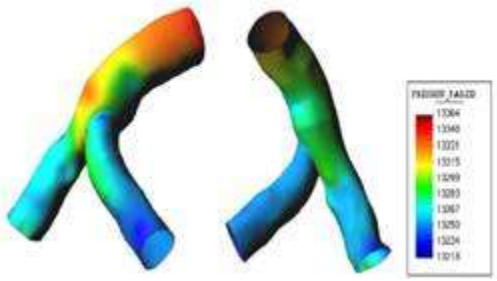


Figura 2: Contornos de presión en la pared internas de las arterias.

<u>En la ingeniería civil:</u>

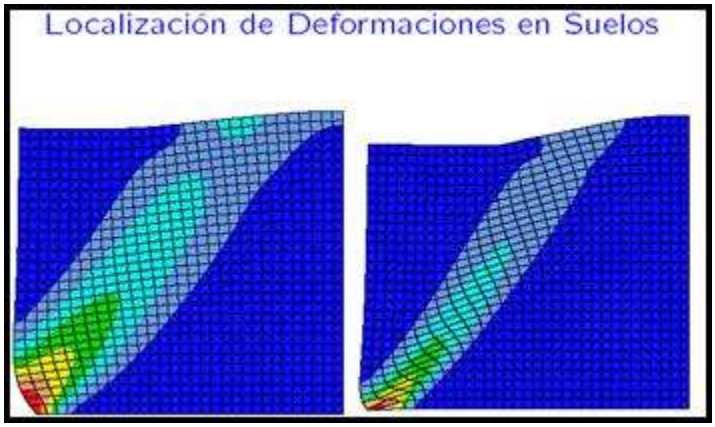


Figura 3: Deformaciones en suelos

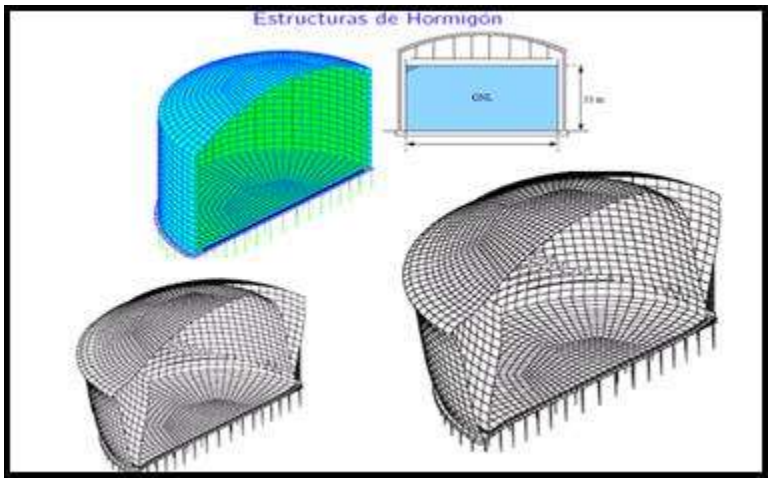


Figura 4: Deformación en tanques de hormigón. Respuesta sísmica por vibración

VII. CONCLUSIONES

Se elaboró el programa del método del elemento finito, para la solución de ecuaciones diferenciales parciales, en el programa MATLAB.

Se utilizó el programa MATLAB, para implementar el algoritmo computacional del método del elemento finito, que optimice el tratamiento de datos en la solución de una ecuación diferencial parcial.

Se desarrolló el algoritmo del elemento finito, para la solución de ecuaciones diferenciales parciales.

ANEXOS

ANEXO N°01: Código correspondiente al 1er ejemplo.

a) PRIMER PROGRAMA.

```
nel=32;
nnel=3;
ndof=1;
nnode=25;
sdof=nnode*ndof;
gcoord(1,1)=0.0; gcoord(1,2)=0.0;
gcoord(2,1)=1.25; gcoord(2,2)=0.0;
gcoord(3,1)=2.5; gcoord(3,2)=0.0;
gcoord(4,1)=3.75; gcoord(4,2)=0.0;
gcoord(5,1)=5.0; gcoord(5,2)=0.0;
gcoord(6,1)=0.0; gcoord(6,2)=2.5;
gcoord(7,1)=1.25; gcoord(7,2)=2.5;
gcoord(8,1)=2.5; gcoord(8,2)=2.5;
gcoord(9,1)=3.75; gcoord(9,2)=2.5;
gcoord(10,1)=5.0; gcoord(10,2)=2.5;
gcoord(11,1)=0.0; gcoord(11,2)=5.0;
gcoord(12,1)=1.25; gcoord(12,2)=5.0;
gcoord(13,1)=2.5; gcoord(13,2)=5.0;
gcoord(14,1)=3.75; gcoord(14,2)=5.0;
gcoord(15,1)=5.0; gcoord(15,2)=5.0;
gcoord(16,1)=0.0; gcoord(16,2)=7.5;
gcoord(17,1)=1.25; gcoord(17,2)=7.5;
gcoord(18,1)=2.5; gcoord(18,2)=7.5;
gcoord(19,1)=3.75; gcoord(19,2)=7.5;
gcoord(20,1)=5.0; gcoord(20,2)=7.5;
gcoord(21,1)=0.0; gcoord(21,2)=10;
gcoord(22,1)=1.25; gcoord(22,2)=10;
gcoord(23,1)=2.5; gcoord(23,2)=10;
gcoord(24,1)=3.75; gcoord(24,2)=10;
gcoord(25,1)=5.0; gcoord(25,2)=10;

nodes(1,1)=1; nodes(1,2)=2; nodes(1,3)=7;
```

```
nodes(2,1)=2; nodes(2,2)=3; nodes(2,3)=8;
nodes(3,1)=3; nodes(3,2)=4; nodes(3,3)=9;
nodes(4,1)=4; nodes(4,2)=5; nodes(4,3)=10;
nodes(5,1)=1; nodes(5,2)=7; nodes(5,3)=6;
nodes(6,1)=2; nodes(6,2)=8; nodes(6,3)=7;
nodes(7,1)=3; nodes(7,2)=9; nodes(7,3)=8;
nodes(8,1)=4; nodes(8,2)=10; nodes(8,3)=9;
nodes(9,1)=6; nodes(9,2)=7; nodes(9,3)=12;
nodes(10,1)=7; nodes(10,2)=8; nodes(10,3)=13;
nodes(11,1)=8; nodes(11,2)=9; nodes(11,3)=14;
nodes(12,1)=9; nodes(12,2)=10; nodes(12,3)=15;
nodes(13,1)=6; nodes(13,2)=12; nodes(13,3)=11;
nodes(14,1)=7; nodes(14,2)=13; nodes(14,3)=12;
nodes(15,1)=8; nodes(15,2)=14; nodes(15,3)=13;
nodes(16,1)=9; nodes(16,2)=15; nodes(16,3)=14;
nodes(17,1)=11; nodes(17,2)=12; nodes(17,3)=17;
nodes(18,1)=12; nodes(18,2)=13; nodes(18,3)=18;
nodes(19,1)=13; nodes(19,2)=14; nodes(19,3)=19;
nodes(20,1)=14; nodes(20,2)=15; nodes(20,3)=20;
nodes(21,1)=11; nodes(21,2)=17; nodes(21,3)=16;
nodes(22,1)=12; nodes(22,2)=18; nodes(22,3)=17;
nodes(23,1)=13; nodes(23,2)=19; nodes(23,3)=18;
nodes(24,1)=14; nodes(24,2)=20; nodes(24,3)=19;
nodes(25,1)=16; nodes(25,2)=17; nodes(25,3)=22;
nodes(26,1)=17; nodes(26,2)=18; nodes(26,3)=23;
nodes(27,1)=18; nodes(27,2)=19; nodes(27,3)=24;
nodes(28,1)=19; nodes(28,2)=20; nodes(28,3)=25;
nodes(29,1)=16; nodes(29,2)=22; nodes(29,3)=21;
nodes(30,1)=17; nodes(30,2)=23; nodes(30,3)=22;
nodes(31,1)=18; nodes(31,2)=24; nodes(31,3)=23;
nodes(32,1)=19; nodes(32,2)=25; nodes(32,3)=24;

bcdof(1)=1;
bcval(1)=0;
bcdof(2)=2;
bcval(2)=0;
bcdof(3)=3;
```

```
bcval(3)=0;
bcdof(4)=4;
bcval(4)=0;
bcdof(5)=5;
bcval(5)=0;
bcdof(6)=6;
bcval(6)=0;
bcdof(7)=11;
bcval(7)=0;
bcdof(8)=16;
bcval(8)=0;
bcdof(9)=21;
bcval(9)=0;
bcdof(10)=22;
bcval(10)=38.2683;
bcdof(11)=23;
bcval(11)=70.7107;
bcdof(12)=24;
bcval(12)=92.3880;
bcdof(13)=25;
bcval(13)=100;


ff=zeros(sdof,1);
kk=zeros(sdof,sdof);
index=zeros(nnel*ndof,1);

for iel=1:nel

nd(1)=nodes(iel,1);
nd(2)=nodes(iel,2);
nd(3)=nodes(iel,3);
x1=gcoord(nd(1),1); y1=gcoord(nd(1),2);
x2=gcoord(nd(2),1); y2=gcoord(nd(2),2);
x3=gcoord(nd(3),1); y3=gcoord(nd(3),2);

index=feeldof(nd,nnel,ndof);
```

```
k=felp2dt3(x1,y1,x2,y2,x3,y3);

kk=feasmbl1(kk,k,index);
end

[kk,ff]=feaplyc2(kk,ff,bcdof,bcval);

fsol=kk\ff;

for i=1:nnode
x=gcoord(i,1); y=gcoord(i,2);
esol(i)=100*sinh(0.31415927*y)*sin(0.31415927*x)/sinh(3.
1415927);
end

num=1:1:sdof;
store=[num' fsol esol']
```

b) **SEGUNDO PROGRAMA:**

```
function [kk]=feasmbl1(kk,k,index)
edof=length(index);
for i=1:edof
ii=index(i);
for j=1:edof
jj=index(j);
kk(ii,jj)=kk(ii,jj)+k(i,j);
end
end
```

C) **TERCER PROGRAMA:**

```
function[index]=feeldof(nd,nnel,ndof)
edof=nnel*ndof;
k=0;
for i=1:nnel
start=(nd(i)-1)*ndof;
for j=1:ndof
```

```
k=k+1;
index(k)=start+j;
end
end
```

D) CUARTO PROGRAMA

```
function [k]=felp2dt3(x1,y1,x2,y2,x3,y3)
A=0.5*(x2*y3+x1*y2+x3*y1-x2*y1-x1*y3-x3*y2);
k(1,1)=((x3-x2)*(x3-x2)+(y2-y3)*(y2-y3))/(4*A);
k(1,2)=((x3-x2)*(x1-x3)+(y2-y3)*(y3-y1))/(4*A);
k(1,3)=((x3-x2)*(x2-x1)+(y2-y3)*(y1-y2))/(4*A);
k(2,1)=k(1,2);
k(2,2)=((x1-x3)*(x1-x3)+(y3-y1)*(y3-y1))/(4*A);
k(2,3)=((x1-x3)*(x2-x1)+(y3-y1)*(y1-y2))/(4*A);
k(3,1)=k(1,3);
k(3,2)=k(2,3);
k(3,3)=((x2-x1)*(x2-x1)+(y1-y2)*(y1-y2))/(4*A);
```

E**) QUINTO PROGRAMA:**

```
function [kk,ff]=feaplyc2(kk,ff,bcdof,bcval)
n=length(bcdof);
sdof=size(kk);
for i=1:n
    c=bcdof(i);
    for j=1:sdof
        kk(c,j)=0;
    end
    kk(c,c)=1;
    ff(c)=bcval(i);
end
```

ANEXO N°02: Código correspondiente al 2º ejemplo.

```
Load plate.txt
[px,py]=gradient(plate);
cs=contour(plate);clabel(cs);hold on
quiver(-px,-py);hold off
```

IX. REFERENCIAS BIBLIOGRÁFICAS:

Alkis Constantinides & Mostoufi, Navid (2000). Numerical Methods for Chemical Engineers with MATLAB Applications". United States of América: EDITORIAL PRENTICE HALL.

Burden, Richard L. & Faires J. Douglas. (2002). Análisis Numérico. (7a ed). México: Inter

Cairó Battistutti, Osvaldo (1996). Metodología de la Programación. (Tomo 1 y 11). México: Alfaomega Grupo Editor, S.A. de C. V.

Carrasco Venegas, Luis (2007). Métodos Numéricos aplicados a la ingeniería. (2a ed). Perú: Ediciones RFG.

Cutipa, C. Miguel & Gutierrez, P. Daniel A. (2007). MATLAB 7. (1a ed). Perú: Editorial macro EIRL.